
--- PAGE 1 of 200 ---

// ===== FILE: backend/prisma/schema.prisma =====

```
generator client {  
  provider = "prisma-client-js"  
}
```

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```
enum UserRole {  
  STUDENT  
  FACULTY  
  ADMIN  
}
```

```
enum ExamStatus {  
  DRAFT  
  SCHEDULED  
  ACTIVE  
  COMPLETED  
  ARCHIVED  
}
```

```
enum SessionStatus {  
  PENDING  
  ACTIVE  
  SUBMITTED  
  TERMINATED  
  INVALIDATED  
}
```

```
enum QuestionType {  
  MULTIPLE_CHOICE  
  MULTIPLE_SELECT  
  TRUE_FALSE  
  SHORT_ANSWER  
  ESSAY  
  FILL_BLANK  
  NUMERICAL  
  CODE  
}
```

```
enum EventType {  
  FACE_NOT_DETECTED  
  MULTIPLE_FACES  
  LOOKING_AWAY  
  TAB_SWITCH  
  WINDOW_BLUR  
  WEBCAM_DENIED  
  NOISE_DETECTED  
---
```

--- PAGE 2 of 200 ---

```
  SCREEN_EXIT  
  COPY_PASTE  
  RIGHT_CLICK  
}
```

```
enum EventSeverity {  
  LOW  
  MEDIUM  
  HIGH  
}
```

```
model User {
```

```

id          String    @id @default(uuid())
email       String    @unique
passwordHash String?
googleId    String?   @unique
role        UserRole
firstName   String
lastName    String
profilePhotoUrl String?
mfaSecret   String?
isVerified  Boolean    @default(false)
isActive    Boolean    @default(true)
lastLoginAt DateTime?
createdAt   DateTime   @default(now())
updatedAt   DateTime   @updatedAt
deletedAt   DateTime?

// Relations
coursesFaculty Course[]    @relation("FacultyCourses")
enrollments      Enrollment[]
examSessions      ExamSession[]
refreshTokens     RefreshToken[]
auditLogs         AuditLog[]
announcements     Announcement[] @relation("AuthoredAnnouncements")
notifications     Notification[]

@@index([email])
@@index([role])
}

model Course {
  id          String    @id @default(uuid())
  code        String    @unique
  name        String
  description  String?
  facultyId   String
  isActive    Boolean    @default(true)
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  deletedAt   DateTime?

  // Relations
  faculty      User          @relation("FacultyCourses", fields: [facultyId], references: [id])
  enrollments  Enrollment[]
}

--- PAGE 3 of 200 ---

  exams        Exam[]
  questions     Question[]
  announcements  Announcement[]

  @@index([facultyId])
  @@index([code])
}

model Enrollment {
  id          String    @id @default(uuid())
  courseId    String
  studentId   String
  enrolledAt  DateTime   @default(now())
  droppedAt   DateTime?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  // Relations
  course Course @relation(fields: [courseId], references: [id])
  student User  @relation(fields: [studentId], references: [id])

  @@unique([courseId, studentId])
  @@index([studentId])
  @@index([courseId])
}

model Announcement {

```

```

id          String    @id @default(uuid())
courseId    String
authorId    String
title       String
body        String
isPinned    Boolean   @default(false)
createdAt   DateTime  @default(now())
updatedAt   DateTime  @updatedAt

// Relations
course Course @relation(fields: [courseId], references: [id], onDelete: Cascade)
author User   @relation("AuthoredAnnouncements", fields: [authorId], references: [id])

@@index([courseId])
}

model Exam {
  id          String    @id @default(uuid())
  courseId    String
  title       String
  instructions String?
  durationMinutes Int
  startAt     DateTime
  endAt       DateTime
  proctoringLevel String @default("STANDARD")
  status      ExamStatus @default(DRAFT)
  isPublished Boolean   @default(false)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  deletedAt   DateTime?

  // Relations
  course      Course      @relation(fields: [courseId], references: [id])
  examQuestions ExamQuestion[]
  examSessions ExamSession[]
  examRules    ExamRules?

  @@index([courseId])
  @@index([status])
  @@index([startAt])
}

model ExamRules {
  id          String    @id @default(uuid())
  examId      String    @unique
  shuffleQuestions Boolean @default(true)
  shuffleChoices Boolean @default(true)
  maxAttempts Int       @default(1)
  negativeMarkingFactor Float @default(0)
  passThreshold Float    @default(60)
  allowCalculator Boolean @default(false)
  allowFormulaSheet Boolean @default(false)

  // Relations
  exam Exam @relation(fields: [examId], references: [id], onDelete: Cascade)
}

model Question {
  id          String    @id @default(uuid())
  courseId    String
  type        QuestionType
  content      Json
  points       Float
  difficulty   String?
  topicTags    String[]
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt
  deletedAt    DateTime?

  // Relations

```

```

course      Course      @relation(fields: [courseId], references: [id])
examQuestions ExamQuestion[]
answers      Answer[]

@@index([courseId])
@@index([type])
}

model ExamQuestion {
  id          String @id @default(uuid())
  examId      String
  questionId  String
  orderIndex  Int
  --- PAGE 5 of 200 ---

  // Relations
  exam      Exam      @relation(fields: [examId], references: [id], onDelete: Cascade)
  question  Question @relation(fields: [questionId], references: [id])

  @@unique([examId, questionId])
  @@index([examId])
}

model ExamSession {
  id          String      @id @default(uuid())
  examId      String
  studentId   String
  startedAt   DateTime    @default(now())
  submittedAt DateTime?
  status      SessionStatus @default(PENDING)
  ipAddress   String?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  // Relations
  exam      Exam      @relation(fields: [examId], references: [id])
  student   User       @relation(fields: [studentId], references: [id])
  answers   Answer[]
  suspiciousEvents SuspiciousEvent[]
  result    Result?

  @@index([examId])
  @@index([studentId])
  @@index([status])
}

model Answer {
  id          String @id @default(uuid())
  sessionId   String
  questionId  String
  response    Json
  autoScore   Float?
  manualScore Float?
  gradedBy    String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relations
  session ExamSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  question Question    @relation(fields: [questionId], references: [id])

  @@unique([sessionId, questionId])
  @@index([sessionId])
}

model SuspiciousEvent {
  id          String      @id @default(uuid())
  sessionId   String
  type        EventType
  --- PAGE 6 of 200 ---

```

```

severity      EventSeverity
screenshotUrl String?
timestamp     DateTime      @default(now())
dismissedBy   String?

// Relations
session ExamSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)

@@index([sessionId])
@@index([severity])
}

model Result {
  id          String   @id @default(uuid())
  sessionId   String   @unique
  totalScore  Float
  percentage  Float
  rank        Int?
  percentile  Float?
  passStatus  Boolean
  isPublished Boolean @default(false)
  finalizedAt DateTime?

  // Relations
  session ExamSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)
}

model RefreshToken {
  id          String   @id @default(uuid())
  userId      String
  tokenHash   String   @unique
  expiresAt   DateTime
  revoked     Boolean   @default(false)
  createdAt   DateTime @default(now())

  // Relations
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([tokenHash])
}

model AuditLog {
  id          String   @id @default(uuid())
  userId      String
  action      String
  resourceType String
  resourceId  String?
  ipAddress   String?
  timestamp   DateTime @default(now())

  // Relations
  user User @relation(fields: [userId], references: [id])

  @@index([userId])
}

--- PAGE 7 of 200 ---

  @@index([timestamp])
}

model Notification {
  id          String   @id @default(uuid())
  userId      String
  type        String   @default("ANNOUNCEMENT")
  title       String
  body        String
  referenceId String?
  isRead      Boolean   @default(false)
  createdAt   DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

```

```

    @@index([userId, isRead])
    @@index([createdAt])
  }
}

// ===== FILE: backend/src/controllers/aiQuestion.controller.ts =====

/**
 * Procto: AI-powered online exam proctoring system.
 * Copyright (c) 2026 Bhagyasree Roy. All rights reserved.
 */

import { Response } from 'express';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { PrismaClient } from '@prisma/client';
import { z } from 'zod';
import { AuthRequest } from '../middleware/auth.middleware';
import { handleError } from '../utils/errors';

const prisma = new PrismaClient();

const generateSchema = z.object({
  courseId: z.string().uuid(),
  topic: z.string().min(2).max(200),
  count: z.number().int().min(1).max(20),
  questionTypes: z
    .array(
      z.enum([
        'MULTIPLE_CHOICE',
        'MULTIPLE_SELECT',
        'TRUE_FALSE',
        'SHORT_ANSWER',
        'FILL_BLANK',
        'NUMERICAL',
      ])
    )
    .min(1),
  difficulty: z.enum(['EASY', 'MEDIUM', 'HARD']),
});

// =====
--- PAGE 8 of 200 ---

// Helpers
// =====

function buildPrompt(
  topic: string,
  count: number,
  questionTypes: string[],
  difficulty: string
): string {
  const typeDescriptions: Record<string, string> = {
    MULTIPLE_CHOICE:
      'Multiple choice (exactly 4 options, exactly one correct answer stored in correctAnswer as a string)',
    MULTIPLE_SELECT:
      'Multiple select (4-5 options, 2-3 correct answers stored in correctAnswer as a JSON array of strings)',
    TRUE_FALSE:
      'True or False (no options array, correctAnswer must be exactly "true" or "false")',
    SHORT_ANSWER:
      'Short answer (no options array, correctAnswer is a concise 1-3 word answer)',
    FILL_BLANK:
      'Fill in the blank (the question text contains a blank shown as ___, correctAnswer is the word or phr...',
    NUMERICAL:
      'Numerical (correctAnswer is a number stored as a string, optionally include tolerance)',
  };

  const typesText = questionTypes
    .map((t) => ` - ${typeDescriptions[t] || t}`)
    .join('\n');

  return `You are an expert educator. Generate exactly ${count} exam question(s) about the topic: "${topic}".`
}

```

Difficulty level: \${difficulty}

Use ONLY these question types (mix them if multiple are listed):
\${typesText}

Return ONLY a valid JSON array with no markdown, no code fences, no extra text. Each element must have this...

```
{
  "type": "<TYPE>",          // one of the allowed types above, in SCREAMING_SNAKE_CASE
  "content": {
    "question": "<string>",    // the question text
    "options": [<string>],    // REQUIRED for MULTIPLE_CHOICE and MULTIPLE_SELECT; omit for others
    "correctAnswer": "<string | string[]>",
    "explanation": "<string>"   // a brief explanation of the correct answer
  },
  "points": <number>,        // EASY=1, MEDIUM=2, HARD=3
  "difficulty": "${difficulty}",
  "topicTags": [<string>]    // 1-3 relevant tags
}
```

Rules:

- For MULTIPLE_CHOICE always include exactly 4 options and correctAnswer as a plain string matching one opt...
- For MULTIPLE_SELECT always include 4+ options and correctAnswer as a JSON array.
- For TRUE_FALSE correctAnswer must be "true" or "false" (lowercase string), no options array.
- Do NOT include any text outside the JSON array.

--- PAGE 9 of 200 ---

- Ensure all JSON is valid and parseable.`;
}

```
function sanitizeGeneratedQuestion(q: any, courseId: string) {
  // Map type aliases the model might return
  const validTypes = new Set([
    'MULTIPLE_CHOICE',
    'MULTIPLE_SELECT',
    'TRUE_FALSE',
    'SHORT_ANSWER',
    'FILL_BLANK',
    'NUMERICAL',
  ]);

  const type = (q.type ?? '').toString().toUpperCase().replace(/ /g, '_');
  if (!validTypes.has(type)) return null;

  const content = q.content ?? {};
  if (!content.question || typeof content.question !== 'string') return null;

  // Normalise correctAnswer for TRUE_FALSE
  if (type === 'TRUE_FALSE') {
    const ca = (content.correctAnswer ?? '').toString().toLowerCase().trim();
    if (ca !== 'true' && ca !== 'false') return null;
    content.correctAnswer = ca;
    delete content.options;
  }

  return {
    courseId,
    type,
    content: {
      question: content.question.trim(),
      options: Array.isArray(content.options)
        ? (content.options as string[]).map((o: string) => o.trim())
        : undefined,
      correctAnswer: content.correctAnswer,
      explanation: content.explanation ?? '',
    },
    points:
      typeof q.points === 'number' && q.points > 0 ? q.points : 1,
    difficulty: ['EASY', 'MEDIUM', 'HARD'].includes(q.difficulty)
      ? q.difficulty
      : 'MEDIUM',
    topicTags: Array.isArray(q.topicTags)
  }
}
```

```
? (q.topicTags as string[]).slice(0, 5)
: [],
};
}

// ████████████████████████████████████████████████████████████████████████████
// Controller
// ████████████████████████████████████████████████████████████████████████████

export const generateQuestionsWithAI = async (
--- PAGE 10 of 200 ---

req: AuthRequest,
res: Response
) => {
try {
    // Validate input
    const data = generateSchema.parse(req.body);

    // Verify faculty owns the course
    if (req.user!.role === 'FACULTY') {
        const course = await prisma.course.findUnique({
            where: { id: data.courseId, facultyId: req.user!.userId },
        });
        if (!course) {
            return res
                .status(403)
                .json({ error: 'Not authorized for this course' });
        }
    }

    // Check API key
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) {
        return res.status(500).json({
            error: 'AI generation is not configured. Please add GEMINI_API_KEY to your .env file.',
        });
    }

    // Call Gemini
    const genAI = new GoogleGenerativeAI(apiKey);
    const model = genAI.getGenerativeModel({ model: 'gemini-flash-latest' });

    const questions: any[] = [];
    let attempts = 0;

    // Retry a few times to fill shortfalls after sanitization/parsing.
    while (questions.length < data.count && attempts < 3) {
        const remaining = data.count - questions.length;
        const prompt = buildPrompt(
            data.topic,
            remaining,
            data.questionTypes,
            data.difficulty
        );

        const result = await model.generateContent(prompt);
        const rawText = result.response.text().trim();

        // Strip any accidental markdown fences the model may prepend
        const jsonText = rawText
            .replace(/^```(?:json)?\s*/i, '')
            .replace(/\s*```\$/, '')
            .trim();

        let parsed: any[];
        try {
```

--- [... pages 11-190 omitted for filing per Rule 70(5) convention ...] ---

Signature of Applicant: _____

--- PAGE 191 of 200 ---

```
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;

// Draw current video frame to canvas
context.drawImage(video, 0, 0, canvas.width, canvas.height);

// Convert to base64
const imageData = canvas.toDataURL('image/jpeg', 0.8);

// Send to backend
try {
  await api.post(`/exam-sessions/${currentSessionId}/webcam-capture`, {
    imageData,
    timestamp: new Date().toISOString(),
  });
} catch (error) {
  console.error('Failed to upload webcam snapshot:', error);
}
};

const sendEvent = async (sessionId: string, type: string, description: string) => {
  try {
    await api.post(`/exam-sessions/${sessionId}/suspicious-event`, {
      type,
      description,
      timestamp: new Date().toISOString(),
    });
  } catch (error: any) {
    console.error('[Proctor] Failed to record event:', type,
      error.response?.status, error.response?.data);
  }
};

const logSuspiciousEvent = (type: string, description: string) => {
  setSuspiciousEvents((prev) => [...prev, type]);

  const currentSessionId = sessionIdRef.current;
  if (!currentSessionId) {
    // Session not ready yet, queue and flush when session starts
    pendingEventsRef.current.push({ type, description });
    return;
  }

  sendEvent(currentSessionId, type, description);
};

const loadExamAndStartSession = async () => {
  try {
    const examResponse = await api.get(`/exams/${examId}`);
    const examData = examResponse.data.exam;
    setExam(examData);

    const sessionResponse = await api.post(`/exam-sessions`, {
      examId: examId,
    });
  }
};
```

--- PAGE 192 of 200 ---

```
const newSessionId = sessionResponse.data.session.id;
sessionIdRef.current = newSessionId;
setSessionId(newSessionId);
setTimeRemaining(examData.durationMinutes * 60);

// Flush any events that fired before session was ready
if (pendingEventsRef.current.length > 0) {
  const queued = pendingEventsRef.current.splice(0);
  queued.forEach(({ type, description }) => sendEvent(newSessionId, type, description));
}

toast.success('Exam started! Good luck!');
```

```

    } catch (error: any) {
      const message = error.response?.data?.error || 'Failed to load exam';
      toast.error(message);
      setTimeout(() => navigate('/student'), 2000);
    }
  };

const saveAnswers = async () => {
  if (!sessionId) return;

  try {
    await api.post(`/exam-sessions/${sessionId}/answers`, {
      answers: Object.entries(answers).map(([questionId, response]) => ({
        questionId,
        response,
      })),
    });
  } catch (error) {
    console.error('Auto-save failed:', error);
  }
};

const handleAnswerChange = (questionId: string, value: any) => {
  setAnswers((prev) => ({
    ...prev,
    [questionId]: value,
  }));
};

const handleAutoSubmit = async () => {
  toast.error('Time is up! Submitting exam...');
  await submitExam();
};

const submitExam = async () => {
  if (!sessionId) return;

  setSubmitting(true);
  try {
    await saveAnswers();
    await api.post(`/exam-sessions/${sessionId}/submit`);
  } catch (error: any) {
    const message = error.response?.data?.error || 'Failed to submit exam';
    toast.error(message);
  } finally {
    setSubmitting(false);
  }
};

const formatTime = (seconds: number) => {
  const hours = Math.floor(seconds / 3600);
  const minutes = Math.floor((seconds % 3600) / 60);
  const secs = seconds % 60;

```

--- PAGE 193 of 200 ---

```

  submittedRef.current = true;
  setSubmitted(true); // Unlock navigation guards (ref is checked by closures, state by JSX)

  // Stop webcam
  if (webcamStream) {
    webcamStream.getTracks().forEach(track => track.stop());
  }

  // Exit fullscreen
  if (document.fullscreenElement) {
    await document.exitFullscreen();
  }

  toast.success('Exam submitted successfully!');
  setTimeout(() => navigate('/student'), 2000);
} catch (error: any) {
  const message = error.response?.data?.error || 'Failed to submit exam';
  toast.error(message);
} finally {
  setSubmitting(false);
}
};

const formatTime = (seconds: number) => {
  const hours = Math.floor(seconds / 3600);
  const minutes = Math.floor((seconds % 3600) / 60);
  const secs = seconds % 60;

```

```

    if (hours > 0) {
      return `${hours}:${minutes.toString().padStart(2, '0')}:${secs.toString().padStart(2, '0')}`;
    }
    return `${minutes}:${secs.toString().padStart(2, '0')}`;
  };

const getTimeColor = () => {
  if (!exam) return 'text-gray-600';
  const totalSeconds = exam.durationMinutes * 60;
  const percentage = (timeRemaining / totalSeconds) * 100;

  if (percentage > 50) return 'text-green-600';
  if (percentage > 25) return 'text-yellow-600';
  return 'text-red-600';
};

const renderQuestion = (question: Question) => {
  const q = question.question;
  const currentAnswer = answers[q.id];

  switch (q.type) {
    case 'MULTIPLE_CHOICE':
      return (
        <div className="space-y-3">
          {q.content.options.map((option: string, index: number) => (
            <label
              key={index}
              className={`flex items-start gap-3 p-4 border-2 rounded-lg cursor-pointer transition ${curr...
                ? 'border-blue-500 bg-blue-50'
                : 'border-gray-200 hover:border-gray-300'
              }`}
            >
              <input
                type="radio"
                name={q.id}
                value={option}
                checked={currentAnswer === option}
                onChange={(e) => handleAnswerChange(q.id, e.target.value)}
                className="mt-1"
              />
              <span className="flex-1">
                {String.fromCharCode(65 + index)}. {option}
              </span>
            </label>
          ))}
        </div>
      );
    case 'MULTIPLE_SELECT':
      return (
        <div className="space-y-3">
          {q.content.options.map((option: string, index: number) => {
            const selectedOptions = currentAnswer || [];
            return (
              <label
                key={index}
                className={`flex items-start gap-3 p-4 border-2 rounded-lg cursor-pointer transition ${se...
                  ? 'border-blue-500 bg-blue-50'
                  : 'border-gray-200 hover:border-gray-300'
                }`}
              >
                <input
                  type="checkbox"
                  checked={selectedOptions.includes(option)}
                  onChange={(e) => {
                    const newSelection = e.target.checked
                      ? [...selectedOptions, option]
                      : selectedOptions.filter((o: string) => o !== option);
                    handleAnswerChange(q.id, newSelection);
                  }}
                />
                {option}
              </label>
            );
          })}
        </div>
      );
  }
};

```

```

        className="mt-1"
      />
      <span className="flex-1">
        {String.fromCharCode(65 + index)}. {option}
      </span>
    </label>
  );
  })}
</div>
);

case 'TRUE_FALSE':
--- PAGE 195 of 200 ---

return (
  <div className="space-y-3">
    {[ 'true', 'false' ].map((option) => (
      <label
        key={option}
        className={`flex items-center gap-3 p-4 border-2 rounded-lg cursor-pointer transition ${cur...
          ? 'border-blue-500 bg-blue-50'
          : 'border-gray-200 hover:border-gray-300'
        }`}
      >
        <input
          type="radio"
          name={q.id}
          value={option}
          checked={currentAnswer === option}
          onChange={(e) => handleAnswerChange(q.id, e.target.value)}
        />
        <span className="flex-1 capitalize font-medium">{option}</span>
      </label>
    )]}
  </div>
);

case 'SHORT_ANSWER':
return (
  <input
    type="text"
    value={currentAnswer || ''}
    onChange={(e) => handleAnswerChange(q.id, e.target.value)}
    className="w-full px-4 py-3 border-2 border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-50..."
    placeholder="Type your answer..."
  />
);

case 'ESSAY':
return (
  <textarea
    value={currentAnswer || ''}
    onChange={(e) => handleAnswerChange(q.id, e.target.value)}
    className="w-full px-4 py-3 border-2 border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-50..."
    rows={8}
    placeholder="Write your answer here..."
  />
);

default:
return <p className="text-gray-500">Question type not supported</p>;
}
};

if (!exam) {
return (
  <div className="min-h-screen bg-gray-50 flex items-center justify-center">
    <div className="text-center">
      <div className="text-6xl mb-4">■</div>
--- PAGE 196 of 200 ---

      <p className="text-xl text-gray-600">Loading exam...</p>

```

```

        </div>
    </div>
    );
}

const currentQuestion = exam.examQuestions[currentQuestionIndex];
const progress = ((currentQuestionIndex + 1) / exam.examQuestions.length) * 100;
const answeredCount = Object.keys(answers).length;

// Face status helpers
const faceStatusDot = () => {
    if (!faceStatus.modelLoaded) return { color: 'bg-gray-400', label: 'AI Loading...' };
    if (faceStatus.faceCount === 0) return { color: 'bg-red-500', label: 'No Face' };
    if (faceStatus.faceCount > 1) return { color: 'bg-orange-500', label: `${faceStatus.faceCount} Faces` };
    return { color: 'bg-green-500', label: 'Face OK' };
};
const dot = faceStatusDot();

return (
    <div className="min-h-screen bg-gray-50">
        <Toaster position="top-right" />

        {/* Webcam feed, small overlay in bottom-right */}
        <div className="fixed bottom-4 right-4 z-30 rounded-xl overflow-hidden shadow-xl border-2 border-gray...
            <video
                ref={videoRef}
                autoPlay
                muted
                playsInline
                className="w-full block"
                style={{ maxHeight: 135, objectFit: 'cover' }}
            />
            {/* Status bar */}
            <div className="flex items-center gap-1.5 px-2 py-1 bg-gray-900">
                <span className={`w-2 h-2 rounded-full ${dot.color} ${faceStatus.faceCount === 1 ? 'animate-pulse...'}` />
                <span className="text-xs text-gray-300 font-medium">{dot.label}</span>
                {!faceStatus.modelLoaded && (
                    <span className="ml-auto text-[0.6rem] text-gray-500">AI init...</span>
                )}
            </div>
        </div>

        {/* Canvas for snapshots (always hidden) */}
        <canvas ref={canvasRef} className="hidden" />

        {/* Face violation warning banner */}
        {faceWarning && (
            <div className="fixed top-0 left-0 right-0 z-40 flex items-center justify-center gap-3 px-4 py-2.5 ...
                <span>{faceWarning}</span>
                <button onClick={() => setFaceWarning(null)} className="ml-4 text-white/70 hover:text-white text-...
            </div>
        )}

        {/* Top Bar */}
    </div>
);

```

--- PAGE 197 of 200 ---

```

<div className="bg-white shadow-sm border-b sticky top-0 z-10">
    <div className="max-w-7xl mx-auto px-4 py-4 flex justify-between items-center">
        <div>
            <h1 className="text-xl font-bold text-gray-900">{exam.title}</h1>
            <p className="text-sm text-gray-600">{exam.course.name}</p>
        </div>

        <div className="flex items-center gap-6">
            {/* Monitoring indicators */}
            <div className="flex items-center gap-2 text-xs">
                <span className="flex items-center gap-1">
                    <span className="w-2 h-2 bg-green-500 rounded-full animate-pulse"></span>
                    Webcam Active
                </span>
                {suspiciousEvents.length > 0 && (
                    <span className="flex items-center gap-1 text-red-600">

```

```

        <span className="w-2 h-2 bg-red-500 rounded-full"></span>
        {suspiciousEvents.length} Event{suspiciousEvents.length > 1 ? 's' : ''}
      </span>
    )}
  </div>

  <div className="text-right">
    <p className="text-sm text-gray-600">Time Remaining</p>
    <p className={`text-2xl font-bold ${getTimeColor()}`}>
      {formatTime(timeRemaining)}
    </p>
  </div>

  <button
    onClick={() => setShowSubmitConfirm(true)}
    disabled={submitting}
    className="px-6 py-3 bg-green-600 text-white rounded-lg hover:bg-green-700 transition font-se...
  >
    Submit Exam
  </button>
</div>
</div>

<div className="w-full bg-gray-200 h-1">
  <div
    className="bg-blue-600 h-1 transition-all duration-300"
    style={{ width: `${progress}%` }}
  />
</div>
</div>

{/* Main Content */}
<div className="max-w-7xl mx-auto px-4 py-8">
  <div className="grid grid-cols-4 gap-6">
    <div className="col-span-1">
      <div className="bg-white rounded-lg shadow p-4 sticky top-24">
        <h3 className="font-bold mb-4">Questions</h3>
        <div className="space-y-2 mb-4">
          <div className="text-sm text-gray-600">
            Answered: {answeredCount} / {exam.examQuestions.length}
          </div>
        </div>
        <div className="grid grid-cols-5 gap-2">
          {exam.examQuestions.map((q, index) => {
            const isAnswered = !!answers[q.question.id];
            const isCurrent = index === currentQuestionIndex;

            return (
              <button
                key={q.id}
                onClick={() => setCurrentQuestionIndex(index)}
                className={`aspect-square rounded-lg text-sm font-semibold transition ${isCurrent
                  ? 'bg-blue-600 text-white'
                  : isAnswered
                    ? 'bg-green-100 text-green-700 hover:bg-green-200'
                    : 'bg-gray-100 text-gray-700 hover:bg-gray-200'
                }`}
              >
                {index + 1}
              </button>
            );
          })}
        </div>
      </div>
    </div>
    <div className="col-span-3">
      <div className="bg-white rounded-lg shadow p-8">
        <div className="flex justify-between items-start mb-6">
          <div className="flex-1">

```

```

<div className="flex items-center gap-3 mb-4">
  <span className="text-sm font-semibold px-3 py-1 bg-blue-100 text-blue-700 rounded">
    Question {currentQuestionIndex + 1} of {exam.examQuestions.length}
  </span>
  <span className="text-sm font-semibold px-3 py-1 bg-green-100 text-green-700 rounded">
    {currentQuestion.question.points} {currentQuestion.question.points === 1 ? 'point' : ...}
  </span>
  <span className="text-sm font-semibold px-3 py-1 bg-gray-100 text-gray-700 rounded">
    {currentQuestion.question.type.replace(/_/g, ' ')}
  </span>
</div>

<h2 className="text-2xl font-bold text-gray-900 mb-6">
  {currentQuestion.question.content.question}
</h2>
</div>
</div>

<div className="mb-8">
  {renderQuestion(currentQuestion)}
</div>

<div className="flex justify-between items-center pt-6 border-t">
  <button
    onClick={() => setCurrentQuestionIndex((prev) => Math.max(0, prev - 1))}
    disabled={currentQuestionIndex === 0}
    className="px-6 py-3 border border-gray-300 text-gray-700 rounded-lg hover:bg-gray-50 tra...
  >
    ← Previous
  </button>

  <div className="text-sm text-gray-500">
    {answers[currentQuestion.question.id] ? (
      <span className="text-green-600 font-medium">✓ Answered</span>
    ) : (
      <span className="text-gray-400">Not answered</span>
    )}
  </div>

  <button
    onClick={() => setCurrentQuestionIndex((prev) => Math.min(exam.examQuestions.length - 1, ...
    disabled={currentQuestionIndex === exam.examQuestions.length - 1}
    className="px-6 py-3 bg-blue-600 text-white rounded-lg hover:bg-blue-700 transition disab...
  >
    Next →
  </button>
</div>
</div>
</div>
</div>
</div>

{/* Submit Confirmation Modal */}
{showSubmitConfirm && (
  <div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center p-4 z-50">
    <div className="bg-white rounded-2xl shadow-xl w-full max-w-md p-8">
      <h3 className="text-2xl font-bold mb-4">Submit Exam?</h3>
      <p className="text-gray-600 mb-6">
        You have answered <span className="font-bold text-blue-600">{answeredCount}</span> out of{' '}
        <span className="font-bold">{exam.examQuestions.length}</span> questions.
      </p>
      <p>
        {answeredCount < exam.examQuestions.length && (
          <p className="text-yellow-600 mb-6 text-sm">
            ■■ You have {exam.examQuestions.length - answeredCount} unanswered question(s).
          </p>
        )}
        {suspiciousEvents.length > 0 && (
          <p className="text-red-600 mb-6 text-sm">
            ■■ {suspiciousEvents.length} suspicious event(s) detected during your exam.
          </p>
        )}
      </p>
    </div>
  </div>
)

```

```

    })
    <p className="text-gray-600 mb-6">
      Are you sure you want to submit? You cannot change your answers after submission.
    </p>

    <div className="flex gap-3">
      <button
        onClick={() => setShowSubmitConfirm(false)}
        className="flex-1 px-4 py-3 border border-gray-300 text-gray-700 rounded-lg hover:bg-gray-50"
--- PAGE 200 of 200 ---

      >
        Cancel
      </button>
      <button
        onClick={() => {
          setShowSubmitConfirm(false);
          submitExam();
        }}
        disabled={submitting}
        className="flex-1 px-4 py-3 bg-green-600 text-white rounded-lg hover:bg-green-700 disabled:...
      >
        {submitting ? 'Submitting...' : 'Submit Exam'}
      </button>
    </div>
  </div>
</div>
  })
</div>
);
}

```